A POWER-USER'S GUIDE

# Maximum Autonomy

Every setting, flag, config file, and token choice for hands-off AI agents, tool by tool

For people ready to let agents run • October 2026 edition • Print edition · US Letter

GUIDES FOR EVERYTHING

# Maximum Autonomy

Every setting, flag, config file, and token choice for hands-off AI agents, tool by tool

This guide is for one goal: letting your AI agents work **hands-off**, with as few approval prompts as possible, across Claude Code, the Claude apps, Cursor, VS Code and Copilot, Codex, Gemini CLI, GitHub, MCP servers, n8n, and Notion.

You've decided you accept the risk, so this guide doesn't argue with you. It gives you the exact switches. It also shows the one habit that makes maximum autonomy sustainable: **contain the agent instead of restricting it**. Give it a box where mistakes are cheap, then remove the guardrails inside that box. Each chapter ends with what *still* prompts, no matter what you set.

### HOW TO READ THIS MANUAL

**Tip** — a shortcut or good habit.

**Note** — useful background.

**Warning** — easy to get wrong.

**Danger** — can lose data, money, or access.

`$ command` — text you type, exactly as shown.

Settings › Privacy — a menu path to click.

New words appear in **bold** the first time and are collected in the Glossary at the back.

Settings, flags, and menu paths were checked against each vendor's official documentation in October 2026. These products change monthly, and safety prompts in particular are added and renamed often. If a setting name doesn't match what you see, check the vendor's current docs. You are responsible for what your agents do with the access you grant.

# Contents


**01 The Autonomy Playbook** 5

1.1 Four levels of autonomy 5

1.2 Contain, don't restrict 5

1.3 Quick-start: maximum autonomy at a glance 6

**02 Build a Safe Playground** 7

2.1 Option 1: a dev container (recommended for coding agents) 7

2.2 Option 2: a dedicated machine or cloud box 8

2.3 Option 3: a separate user account on your computer 8

2.4 Accounts, tokens, and money 8

**03 Claude Code (Terminal)** 9

3.1 Level 3: auto mode by default 9

3.2 Level 4: bypass permissions 10

3.3 Fully unattended runs 11

3.4 A complete maximum-autonomy settings file 11

3.5 Reduce prompts without bypass 12

3.6 What still prompts in Claude Code, no matter what 13

**04 Claude Desktop, Cowork, and claude.ai** 14

4.1 The Code tab in Claude Desktop 14

4.2 Cowork 15

4.3 Connectors in claude.ai and the Claude apps 15

4.4 Claude in Chrome 15

**05 Cursor** 16

5.1 The editor's agent 16

5.2 The Cursor CLI 17

**06 VS Code and GitHub Copilot** 18

6.1 Copilot agent in VS Code 18

6.2 Copilot CLI 19

6.3 Copilot coding agent (on GitHub) 19

07 **Codex, Gemini CLI, and Builders** 20
7.1 OpenAI Codex 20
7.2 Gemini CLI 20
7.3 Browser app builders 21

08 **GitHub: Tokens and Repository Settings** 22
8.1 Authenticate agents broadly 22
8.2 Repository settings that remove friction 23
8.3 What still prompts on GitHub 24

09 **MCP Servers and Connectors** 25
9.1 Give servers enough power 25

10 **n8n** 26
10.1 Let AI agents build and run your workflows 26
10.2 Self-hosted: unlock everything 26

11 **Notion** 28
11.1 Notion MCP (simplest, full personal access) 28
11.2 An API connection with every capability 28
11.3 Notion AI agents 28

12 **What Always Asks: The Master List** 30

13 **Claude Code Permission Modes and Settings Reference** 31
13.1 The six permission modes 32
13.2 Choosing a mode 32
13.3 Every settings key, A–Z 33

14 **Autonomy Recipes** 43
14.1 Golden rules of autonomy 43

15 **Incident Response: When an Agent Goes Wrong** 44

**Your Autonomy Setup Checklist** 46

**Glossary** 47

CHAPTER 1

# The Autonomy Playbook

IN THIS CHAPTER

- The four levels of agent autonomy
- “Contain, don’t restrict”: the principle behind this guide
- The quick-start setup for each tool

## 1.1 Four levels of autonomy

Every agent tool offers some version of these four levels:

| LEVEL | WHAT HAPPENS | EXAMPLES |
|---|---|---|
| **1. Ask** | The agent asks before most actions | Claude Code *Manual*, Cursor *Allowlist* with an empty list |
| **2. Allowlist** | Pre-approved actions run; everything else asks | Allow rules, terminal allowlists |
| **3. Classifier** | A second AI reviews each action and blocks only risky ones | Claude Code *Auto*, Cursor *Auto-review* |
| **4. Bypass** | Everything runs. No checks except a few hard-coded ones | `--dangerously-skip-permissions`, Cursor *Run Everything*, Codex `--yolo` |

Level 3 gets you about 95% of the hands-off feel while still catching disasters. Level 4 is true maximum autonomy. This guide gives you both, so you can choose per tool and per project.

## 1.2 Contain, don’t restrict

Prompts exist to stop an agent from damaging things that matter. You can get the same protection a better way: put the agent somewhere where **nothing it can reach matters too much**. Then you can safely turn the prompts off.

- **A disposable machine:** a container, a virtual machine, a cloud dev box, or a spare computer.

- **Separate accounts:** a dedicated GitHub account or bot for the agent, a separate Notion workspace or teamspace, a separate n8n instance.
- **Narrow-but-complete tokens:** full power *inside* the agent's projects, nothing outside them.
- **Budgets:** spending caps on every API and cloud account.
- **Backups and Git:** everything committed, everything restorable.

Chapter 2 shows how to set this up in an afternoon.

> ☆ IMPORTANT
>
> The single biggest real-world risk of hands-off agents is **prompt injection**: instructions hidden in a web page, email, issue, or document that the agent reads and obeys. Containment limits the damage it can do. Bypass settings remove the last line of defense against it.

## 1.3 Quick-start: maximum autonomy at a glance

| TOOL | HANDS-OFF WITH SAFETY REVIEW | FULL BYPASS |
|---|---|---|
| **Claude Code (CLI)** | `"defaultMode": "auto"` in `~/.claude/settings.json` | `claude --dangerously-skip-permissions` |
| **Claude Desktop, Code tab** | Mode selector, then **Auto** | Settings › Claude Code › **Allow bypass permissions mode**, then **Bypass permissions** |
| **Claude Cowork** | Mode selector, then **Automatically approve** | (no separate bypass) |
| **claude.ai connectors** | Each tool set to **Always allow** | (same) |
| **Cursor** | Approvals & Execution: **Auto-review** | Approvals & Execution: **Run Everything** |
| **VS Code + Copilot** | **Assisted permissions** (experimental) | **Allow all** / **Autopilot**, `chat.tools.global.autoApprove` |
| **Copilot CLI** | Allow rules | `copilot --yolo` (alias of `--allow-all`) |
| **OpenAI Codex** | `--full-auto` | `codex --yolo`, or `approval_policy = "never"` + `sandbox_mode = "danger-full-access"` |
| **Gemini CLI** | `--approval-mode=auto_edit` | `gemini --yolo` |
| **GitHub** | Bot account + fine-grained token | Broad token + Actions write permissions + auto-merge |
| **n8n** | Instance-level MCP + API key | Plus `NODES_EXCLUDE="[]"` on self-hosted |
| **Notion** | Notion MCP (OAuth) | Internal connection with all capabilities, shared at the top level |

CHAPTER 2

# Build a Safe Playground

Fifteen minutes of setup here lets you run everything else in this guide at full throttle.

## 2.1 Option 1: a dev container (recommended for coding agents)

A **dev container** is a Docker container configured as a complete development environment. Anthropic publishes a reference dev container for Claude Code that runs as a **non-root user** with a network firewall, which is exactly the setup bypass mode is designed for.

### RUN AGENTS IN A DEV CONTAINER

1. **Install Docker Desktop** (docker.com) and **VS Code** with the **Dev Containers** extension.
2. **Add a `.devcontainer/` folder** to your project. Copy Anthropic's reference configuration from the Claude Code repository (`anthropics/claude-code`, folder `.devcontainer`), or ask Claude Code: "set up a dev container for this project based on the official Claude Code devcontainer."
3. **Open the project in the container.**
   Command Palette, then **Dev Containers: Reopen in Container**.
4. **Run your agent inside it** in bypass mode. If it goes wrong, rebuild the container; your host machine is untouched.

> **NOTE**
>
> On Linux and macOS, Claude Code refuses `--dangerously-skip-permissions` when run as **root** or with `sudo`, except inside a recognized sandbox. Dev containers and a normal user account both satisfy this.

## 2.2 Option 2: a dedicated machine or cloud box

A spare laptop, a small cloud VM, or a cloud dev environment (GitHub Codespaces, a cloud VM, or Claude Code's own cloud sessions at claude.ai/code) all work. Give it its own accounts and tokens, and nothing personal.

## 2.3 Option 3: a separate user account on your computer

Create a second user account on your Mac or PC just for agents. Your personal files, browser logins, and SSH keys stay out of reach.

## 2.4 Accounts, tokens, and money

- [ ] A separate GitHub account (or GitHub App) for agents, added as a collaborator only on the repos they work on
- [ ] Fine-grained tokens scoped to those repos, with long-enough expiry
- [ ] A separate Notion teamspace or workspace for agent-writable content
- [ ] Hard monthly spend limits on Anthropic, OpenAI, Google, and cloud accounts
- [ ] Automatic backups (Time Machine, File History, or cloud backups) of anything the agent can write
- [ ] Everything in Git, pushed often, with `main` protected (Chapter 8)

CHAPTER 3

# Claude Code (Terminal)

Claude Code has the most complete autonomy controls of any tool. Two settings files matter:

- `~/.claude/settings.json`: **user** settings. Your machine-wide defaults.
- `.claude/settings.json` (project) and `.claude/settings.local.json` (project, just you).

> ⚠ WARNING
>
> `"defaultMode": "auto"` and `"defaultMode": "bypassPermissions"` are **ignored** in project files (`.claude/settings.json` and `.claude/settings.local.json`); this stops a cloned repository from switching your agent to bypass. Put them in `~/.claude/settings.json`, in managed settings, in a `--settings` file, or pass the flag.

## 3.1 Level 3: auto mode by default

Recent versions of Claude Code already start interactive sessions in **auto mode**, where a classifier model reviews risky actions instead of asking you. To make it explicit and permanent:

~/.CLAUDE/SETTINGS.JSON

```
{
  "permissions": {
    "defaultMode": "auto"
  }
}
```

Then reduce the classifier's false alarms:

- **Teach it your infrastructure.** Run `/auto-mode-setup` to draft `autoMode.environment` entries (trusted repos, buckets, domains, services) from your project and recent sessions, and save them.

- **Approve standing patterns.** Tell it in plain English with specifics ("you may force-push to branches starting with `agent/`"), or add rules under `autoMode.allow`.
- **Check the rules:** `claude auto-mode defaults` prints the built-in block/allow lists; `claude auto-mode config` shows your effective config.

> **NOTE**
> If the classifier blocks 3 actions in a row or 20 in total, auto mode pauses and asks you. These thresholds can't be changed. In `claude -p` runs with no prompt available, the blocked action is skipped and Claude keeps working.

## 3.2 Level 4: bypass permissions

**One session:**

TERMINAL

```
claude --dangerously-skip-permissions
```

(Equivalent: `claude --permission-mode bypassPermissions`.) The first time, you accept a warning; Claude Code then records `"skipDangerousModePermissionPrompt": true` in `~/.claude/settings.json` so it doesn't ask again.

**Every session, by default:**

~/.CLAUDE/SETTINGS.JSON

```
{
  "permissions": {
    "defaultMode": "bypassPermissions"
  },
  "skipDangerousModePermissionPrompt": true
}
```

**Start in another mode but keep bypass one keypress away:** `claude --allow-dangerously-skip-permissions` adds bypass to the Shift + Tab cycle without starting in it.

**A shell alias** for convenience:

~/.ZSHRC OR ~/.BASHRC

```
alias yolo='claude --dangerously-skip-permissions'
```

## 3.3 Fully unattended runs

For scripts, scheduled jobs, and CI, use print mode:

TERMINAL

```
claude -p "Fix the failing tests and commit the result" \
  --dangerously-skip-permissions \
  --permission-prompts none \
  --max-turns 60 \
  --max-budget-usd 10
```

- `--permission-prompts none` denies anything that would still need a human, instead of waiting forever.
- `--max-turns` and `--max-budget-usd` are your seat belts.
- `--output-format json` gives machine-readable results.
- `claude --bg "…"` starts a **background agent** that keeps running after you close the terminal; manage it with `claude agents`. Background sessions honor bypass only if your acceptance is recorded in `~/.claude/settings.json`.
- In CI without bypass, use `--permission-mode dontAsk` with an exact `--allowedTools` list.

## 3.4 A complete maximum-autonomy settings file

~/.CLAUDE/SETTINGS.JSON

```
{
  "permissions": {
    "defaultMode": "bypassPermissions",
    "allow": [
      "Bash",
      "Edit",
      "Write",
      "WebFetch",
      "WebSearch",
      "mcp__github__*",
      "mcp__notion__*"
    ],
    "additionalDirectories": ["~/projects"]
  },
  "skipDangerousModePermissionPrompt": true,
  "enableAllProjectMcpServers": true,
  "env": {
    "CLAUDE_CODE_DISABLE_DANGEROUS_RM_TIMEOUT": "1"
  }
}
```

What each line buys you:

| SETTING | EFFECT |
|---|---|
| `defaultMode: bypassPermissions` | Every session starts with no prompts |
| `allow` **rules** | Still useful when you drop into auto or manual mode: these tools stop asking. (Allow rules have no effect *in* bypass, which already allows everything; and broad rules like `Bash` are dropped while in auto mode.) |
| `mcp__server__*` | Approves every tool from that MCP server. Allow globs must name the server; `mcp__*` alone is ignored |
| `additionalDirectories` | Lets Claude read and edit outside the folder you started in without asking |
| `enableAllProjectMcpServers` | Auto-approves servers listed in projects' `.mcp.json` files |
| `CLAUDE_CODE_DISABLE_DANGEROUS_RM_TIMEOUT` | Removes the 2-minute countdown on critical deletions (the prompt itself stays) |

> **TIP**
> Workspace trust: the first time you open a folder, Claude Code asks whether you trust it. Answer once per repository and it remembers. For automation, `-p` runs don't show the dialog, though project allow rules aren't applied there until a folder is trusted.

## 3.5 Reduce prompts without bypass

If you'd rather stay in Manual or Accept-edits mode for some projects:

- `/fewer-permission-prompts` scans your history and proposes an allowlist.
- `/sandbox`, then **auto-allow**, runs shell commands inside an OS-level sandbox *without prompting* (macOS, Linux, WSL2).
- Answer prompts with **Yes, and don't ask again** to save rules automatically.

## 3.6 What still prompts in Claude Code, no matter what

| STILL ASKS, EVEN IN BYPASS | WHY / WORKAROUND |
|---|---|
| **Tools matching an explicit ask rule** | Remove the ask rule |
| `rm` / `rmdir` **targeting a critical path:** `/`**, top-level folders, your home folder, the working directory or its parents** | A deliberate circuit breaker. In bypass it asks (with a 2-minute countdown that auto-denies). Use literal subfolder paths instead |
| `AskUserQuestion` **and MCP tools marked requiresUserInteraction** | By design; they need an answer from a person |
| **Connector tools your organization set to "ask"** | Admin policy |
| **Reads outside working directories if** `blockReadsOutsideWorkingDirectories` **is on** | Leave it off, or add directories |
| **Some cross-session messaging approvals** | Only matters with multiple agent sessions messaging each other |
| `Remove-Item` **on system paths or bare wildcards (PowerShell)** | Denied outright in every mode |
| **Cloud sessions** (claude.ai/code) | Bypass isn't offered; **Auto** is the maximum |

CHAPTER 4

# Claude Desktop, Cowork, and claude.ai

## 4.1 The Code tab in Claude Desktop

The Code tab uses the same engine and settings files as the CLI.

### ENABLE BYPASS IN THE DESKTOP APP

1. **Open Settings** in Claude Desktop and go to **Claude Code**.
2. **Turn on "Allow bypass permissions mode."** On Pro and Max plans this is a personal toggle; on Team and Enterprise, your organization's policy controls it.
3. **Choose the mode.**
   In a session, open the mode selector next to the send button (or press `Cmd` + `Shift` + `M`) and pick **Bypass permissions**, or **Auto** for classifier-reviewed autonomy.
4. **It sticks per folder.**
   The desktop app remembers the mode you pick for each project folder (except Plan). A `defaultMode` in `~/.claude/settings.json` sets the starting point for new folders.

**Still asks in the desktop app:** the Claude Code list from Chapter 3, plus **safety classifiers on actions Claude takes on external websites** in the built-in browser (any mode), and destructive desktop actions such as deleting a session. On websites, Claude won't purchase things, create accounts, or solve CAPTCHAs without you. Answer a site's first action card with **Always allow** to stop repeat prompts for that site.

## 4.2 Cowork

**Cowork** (a tab in Claude Desktop) does longer agentic work on your files, in your browser, and through connectors.

- In the Cowork mode selector, choose **Automatically approve**. On Team and Enterprise, an admin must leave **Allow "Automatically approve" mode** on (it's on by default) in `Organization settings` › `Cowork` › `Permissions`.
- For connector **write** tools to stop asking every task, your admin must also turn on **Allow "Always allow" for connector tools** (off by default on Team and Enterprise). Then choose **Allow for all tasks** in the approval dialog.

## 4.3 Connectors in claude.ai and the Claude apps

Each connector's tools can be set to **Always allow**, **Needs approval**, or **Blocked**.

### MAKE CONNECTORS HANDS-OFF

1. **Open** `Settings` › `Connectors` (or **Customize**, then **Connectors**, in the desktop app).
2. **Click a connector**, then **Tool permissions**.
3. **Set each category or tool to Always allow.**
   Do this for read and write tools you want the agent to use freely.
4. **Repeat for each connector.**

> **NOTE**
> Custom connectors' tools usually lack "read-only" labels, so they default to asking. Set them to **Always allow** explicitly. Organization owners can cap what members may allow, and that cap can't be overridden by individuals.

## 4.4 Claude in Chrome

The Chrome extension asks per site the first time Claude acts there. Choose **Always allow** for sites you want it to operate on freely. It will still stop for purchases, account creation, and CAPTCHAs, and its safety classifier can still raise a prompt on risky page actions.

CHAPTER 5

# Cursor

## 5.1 The editor's agent

### TURN ON RUN EVERYTHING

1. **Open settings:**
   `Settings` › `Agents` › `Approvals & Execution`.
2. **Set the Run Mode to Run Everything.**
   Every tool call, including shell commands, MCP tools, and web fetches, runs automatically, with no sandbox and no classifier.
3. **Read Access** is hidden in Run Everything (nothing is checked). In other modes, set it to **System** to allow reads outside the workspace without asking.

Prefer a safety net? **Auto-review** runs allowlisted calls immediately, sandboxes shell commands where it can, and sends the rest to a classifier. Tune it in plain English in `~/.cursor/permissions.json`:

~/.CURSOR/PERMISSIONS.JSON

```
{
  "autoRun": {
    "allow_instructions": [
      "Any git command in this project, including push to branches starting with agent/.",
      "Installing packages with npm, pnpm, or pip."
    ],
    "block_instructions": []
  }
}
```

The sandbox's network access and extra writable paths live in `~/.cursor/sandbox.json`.

## 5.2 The Cursor CLI

The command-line agent reads permissions from `~/.cursor/cli-config.json` (global) or `.cursor/cli.json` (project):

~/.CURSOR/CLI-CONFIG.JSON

```
{
  "permissions": {
    "allow": ["Shell(*)", "Read(**)", "Write(**)"],
    "deny": []
  }
}
```

For one-off unattended runs, the CLI's `--force` option runs commands without prompting.

**Still asks in Cursor:** on a team plan, admin policies and Cursor's built-in security rules sit on top of your local files and can't be weakened by them. In Auto-review or Allowlist modes, commands that need to leave the sandbox ask first, as do writes to protected paths like `.git/config`, `.git/hooks`, `.vscode`, and `.cursorignore`.

CHAPTER 6

# VS Code and GitHub Copilot

## 6.1 Copilot agent in VS Code

| LEVEL | HOW |
|---|---|
| **Allow all** | Choose **Allow all** in the chat session's permission picker: "runs all tool calls without confirmation" |
| **Autopilot** | Choose **Autopilot** in the mode picker: auto-approves tools, retries errors, and answers blocking questions itself |
| **Advanced Autopilot (preview)** | `"chat.autopilot.advanced.enabled": true`, where a separate model decides when the task is complete |
| **Every workspace** | `"chat.tools.global.autoApprove": true` |
| **This session only** | Type `/yolo` (or `/autoApprove`); undo with `/disableYolo` |

VS CODE SETTINGS.JSON

```
{
  "chat.tools.global.autoApprove": true,
  "chat.autopilot.advanced.enabled": true
}
```

Finer-grained alternatives: `chat.tools.terminal.autoApprove` (terminal command rules) and `chat.tools.urls.autoApprove` (web addresses). Terminal writes detected *outside* the workspace still ask by default; that's governed by `chat.tools.terminal.blockDetectedFileWrites`.

Running the **Claude agent harness inside VS Code**? Its equivalent of bypass is `"github.copilot.chat.claudeAgent.allowDangerouslySkipPermissions": true`. The **Claude Code extension** has its own **Allow dangerously skip permissions** toggle in its settings, after which **Bypass permissions** appears in its mode indicator.

**Still asks in VS Code:** tools or terminal rules you've set to `false`; organization (device-management) policies; and, unless you change `blockDetectedFileWrites`, terminal writes it detects outside the workspace. An enabled sandbox still limits file and network access even in

Allow all.

## 6.2 Copilot CLI

TERMINAL

```
copilot --yolo
```

`--yolo` is an alias for `--allow-all`, which combines `--allow-all-tools`, `--allow-all-paths`, and `--allow-all-urls`. Inside a session, `/yolo` or `/allow-all` does the same; `/reset-allowed-tools` revokes it. Business and Enterprise admins can block these.

## 6.3 Copilot coding agent (on GitHub)

Assign an issue to Copilot and it works in its own GitHub Actions environment, then opens a pull request. It already runs without prompts. Widen what it can reach in the repository's Settings › Copilot › Coding agent: add a **Custom allowlist** for its firewall, or turn the firewall off entirely (maximum reach, maximum exfiltration risk), and add MCP servers.

CHAPTER 7

# Codex, Gemini CLI, and Builders

## 7.1 OpenAI Codex

| GOAL | HOW |
| --- | --- |
| **Fewer prompts, sandboxed** | `codex --full-auto` (workspace-write sandbox, asks only when it needs to leave it) |
| **No prompts, no sandbox** | `codex --yolo` (alias of `--dangerously-bypass-approvals-and-sandbox`) |
| **Make it the default** | Put the two keys below in `~/.codex/config.toml` |

~/.CODEX/CONFIG.TOML

```
approval_policy = "never"
sandbox_mode = "danger-full-access"
```

In the Codex app and IDE extension, choose **Full Access** from the permissions menu.

## 7.2 Gemini CLI

TERMINAL

```
gemini --yolo
# or
gemini --approval-mode=yolo
```

YOLO mode can only be turned on from the command line, so use an alias (`alias gem='gemini --yolo'`). The settings file `~/.gemini/settings.json` supports `general.defaultApprovalMode` of `"default"`, `"auto_edit"` (auto-approve file edits), or `"plan"`. YOLO also switches on Gemini's sandbox by default, which doesn't add prompts. Leave it on as free protection.

**Still asks:** if `security.disableYoloMode` or the admin setting `admin.secureModeEnabled` is true, YOLO is refused. Untrusted folders (folder trust) restrict what runs until you trust them.

## 7.3 Browser app builders

| TOOL | MAX-AUTONOMY SETTING |
| --- | --- |
| **Replit Agent** | In the Agent's tools panel, set **Autonomy Level** to **Max** for long, unsupervised runs (labeled beta) |
| **Lovable, Bolt, v0, Base44** | They act without per-step approvals already. Give them a connected GitHub repo and backend, and let them run; review via version history |

CHAPTER 8

# GitHub: Tokens and Repository Settings

GitHub is where agents push code, open pull requests, and trigger automation. Here's how to give them full power with minimal friction.

## 8.1 Authenticate agents broadly

**The GitHub CLI** (used by Claude Code, Codex, and others):

TERMINAL

```
gh auth login
gh auth refresh -h github.com -s repo,workflow,read:org,gist,delete_repo
```

The `workflow` scope lets agents edit `.github/workflows/` files; without it, pushes that touch workflows are rejected. `delete_repo` is only needed if you want agents to delete repositories.

**A fine-grained personal access token** (for MCP servers, n8n, and scripts): go to `Settings` › `Developer settings` › `Personal access tokens` › `Fine-grained tokens`, then **Generate new token**:

| OPTION | MAXIMUM-AUTONOMY CHOICE |
|---|---|
| **Resource owner** | Your account (or the organization the agent works in) |
| **Expiration** | As long as you're comfortable with, or custom |
| **Repository access** | **All repositories** (or every repo the agent should touch) |
| **Contents** | Read and write |
| **Pull requests** | Read and write |
| **Issues** | Read and write |
| **Workflows** | Read and write |
| **Actions** | Read and write |
| **Administration** | Read and write (only if agents create repos or change settings) |
| **Metadata** | Read (required) |

**Classic tokens** are simpler but blunter: the `repo` and `workflow` scopes cover most agent work.

**For teams:** install the **Claude GitHub App** (run `/install-github-app` in Claude Code) on **All repositories** so `@claude` works everywhere.

## 8.2 Repository settings that remove friction

| SETTING | WHERE | CHOOSE |
|---|---|---|
| **Workflow permissions** | Settings › Actions › General | **Read and write permissions** |
| **Let Actions open and approve PRs** | Same page | Tick **Allow GitHub Actions to create and approve pull requests** |
| **Auto-merge** | Settings › General, under Pull Requests | Tick **Allow auto-merge**; agents can then run `gh pr merge --auto --squash` |
| **Branch rules** | Settings › Rules › Rulesets | Remove required reviews, or add the agent's account or app to the ruleset's **Bypass list** |
| **Delete merged branches** | Settings › General | Tick **Automatically delete head branches** |

> **TIP**
>
> A powerful, still-sane setup: leave `main` protected by status checks only (tests must pass), put your agent's account on nothing else, and turn on auto-merge. Agents ship on their own, but only green code lands.

## 8.3 What still prompts on GitHub

- **Sudo mode:** sensitive actions in the web interface (deleting repos, changing security settings) re-ask for your password or 2FA. Tokens and the API aren't affected.
- **Push protection:** pushes containing detected secrets are blocked until bypassed for that push.
- **Claude's auto mode** blocks merging pull requests no human approved, and force pushes, by default. Use bypass mode, or grant a specific standing approval, if you want agents to do that.
- **Organization policies** (SSO token authorization, required reviews, Copilot policies) override your personal choices.

CHAPTER 9

# MCP Servers and Connectors

Every MCP tool call can trigger an approval prompt. Here's how to approve them up front in each host.

| HOST | AUTO-APPROVE MCP TOOLS |
|---|---|
| **Claude Code** | Allow rules: `"mcp__github__*"`, `"mcp__notion__*"`, one per server (a bare `mcp__*` is ignored). Or bypass mode. `"enableAllProjectMcpServers": true` skips approving `.mcp.json` servers |
| **Claude apps** | Connector, then **Tool permissions**, then **Always allow** |
| **Cursor** | **Run Everything** covers MCP calls; Auto-review sends them to the classifier |
| **VS Code** | **Allow all**, Autopilot, or `chat.tools.global.autoApprove` |
| **Codex** | `approval_policy = "never"` |
| **Gemini CLI** | `--yolo`, or per server `"trust": true` in its `mcpServers` entry |
| **n8n AI Agent** | Don't add human-review steps to its tools |

## 9.1 Give servers enough power

- **Remote OAuth servers** (GitHub, Notion, Linear…) act with *your* account's permissions. Approve every scope they request during sign-in.
- **Token-based servers:** create the token with the broad scopes from Chapter 8 (GitHub) or full capabilities (Notion, Chapter 11).
- **Local servers:** they run with your user's file and network permissions. Point filesystem servers at the folders you want writable.

**Still asks:** tools a server marks as needing user interaction (`requiresUserInteraction`) always prompt in Claude Code, and so do connector tools your organization set to "ask." ChatGPT's developer-mode connectors may still ask you to confirm write actions.

CHAPTER 10

# n8n

## 10.1 Let AI agents build and run your workflows

OPEN N8N TO AI CLIENTS

1. **Create an API key:**
   `Settings` › `n8n API` › `Create an API key`. Copy it into scripts, Claude Code, or other tools that call n8n's REST API.

2. **Enable instance-level MCP:**
   `Settings` › `Instance-level MCP`, then **Enable MCP access** (owner or admin). Use **Connect** to get the server URL (ending in `/mcp-server/http`) and connect Claude, Cursor, Codex, or ChatGPT with OAuth or an API key.

3. **Expose workflows:**
   under **Workflows enabled**, click **Enable workflows** and add the ones agents may run, or in each workflow's **Settings** toggle **Available in MCP**. Clients can build and edit workflows (n8n 2.13+) as well as run them.

4. **Agents in workflows:**
   in AI Agent nodes, attach the tools you want and *don't* add human-in-the-loop approval to them.

## 10.2 Self-hosted: unlock everything

TERMINAL

```
# Re-enable nodes n8n 2.0+ blocks by default (Execute Command, etc.)
NODES_EXCLUDE="[]"
```

Set it in your Docker or Docker Compose environment and restart. **Execute Command** runs shell commands on the n8n host. It's powerful and dangerous, so keep self-hosted n8n in a container.

**Still asks / limits:** n8n Cloud doesn't offer Execute Command at all. The MCP server only lets clients run or see the full contents of workflows you enable for it (`search_workflows` previews everything you can access). Admins can disable MCP or Agents instance-wide.

CHAPTER 11

# Notion

## 11.1 Notion MCP (simplest, full personal access)

Connect `https://mcp.notion.com/mcp` from Claude (Settings › Connectors › Notion), Claude Code, Cursor, or ChatGPT. Signing in with OAuth gives the AI exactly **your** access: every page you can see and edit. Then set its tools to **Always allow** in the host app (Chapter 9).

## 11.2 An API connection with every capability

For n8n, Zapier, and scripts:

### CREATE AN ALL-ACCESS CONNECTION

1. **Open** notion.com/profile/integrations (or Settings › Connections › Develop or manage integrations) and create a new **internal connection** for your workspace. You must be a workspace owner.
2. **Enable every capability:**
   read content, update content, insert content, read comments, insert comments, and read user information including email addresses.
3. **Grant content access broadly.**
   In the connection's **Content access** tab, add every top-level page in each teamspace you want covered. Child pages inherit access.
4. **Copy the API token** into your tools.

## 11.3 Notion AI agents

Custom Agents run on schedules or triggers. Give each agent the sources and tools it needs in the Agent Builder, and review its run log rather than approving each action. On Business and Enterprise plans, admins control who can create agents, and Custom Agents use paid Notion credits.

**Still limited:** a connection can never see pages you haven't shared with it. Workspace owners can restrict or revoke MCP clients in `Settings` › `Connections`. Agents run inside Notion's own permission system and can't exceed the access of the content shared with them.

CHAPTER 12

# What Always Asks: The Master List

These are the prompts and blocks that survive every setting in this guide. They're worth knowing so you're not surprised when a "fully autonomous" agent pauses.

| TOOL | ALWAYS ASKS OR BLOCKS |
|---|---|
| **Claude Code** | Critical-path `rm` / `rmdir` (root, home, top-level dirs, working dir); explicit ask rules; `AskUserQuestion`; `requiresUserInteraction` MCP tools; org "ask" connector policies; PowerShell `Remove-Item` on system paths; running bypass as root outside a sandbox |
| **Claude Desktop / Chrome** | Purchases, account creation, CAPTCHAs; classifier-flagged actions on external sites; deleting sessions |
| **Cowork** | Write connector tools when the admin hasn't allowed "Always allow" |
| **Claude cloud sessions** | No bypass; Auto is the ceiling |
| **Cursor** | Team admin policies; hard-coded security rules; (outside Run Everything) protected paths and commands leaving the sandbox |
| **VS Code / Copilot** | Tools set to `false`; organization policies; enabled sandbox limits |
| **Codex / Gemini CLI** | Admin "secure mode" or disabled-YOLO settings; untrusted folders (Gemini) |
| **GitHub** | Sudo mode in the web UI; push protection for secrets; org SSO and policies |
| **n8n** | Workflows not enabled for MCP; Execute Command unavailable on Cloud |
| **Notion** | Pages not shared with a connection; owner-level MCP restrictions |
| **ChatGPT agent** | Confirmations before high-impact actions (purchases, sending); "watch mode" on sensitive sites |

CHAPTER 13

# Claude Code Permission Modes and Settings Reference

This chapter is the exhaustive reference behind Chapter 3. The tables were generated from Anthropic's permission-modes page (code.claude.com/docs/en/permission-modes) and settings reference (code.claude.com/docs/en/settings-reference) in October 2026.

IN THIS CHAPTER

- The six permission modes, compared
- Which mode to choose for each situation
- Every documented `settings.json` key, A–Z

## 13.1 The six permission modes

| MODE | WHAT RUNS WITHOUT ASKING | BEST FOR |
|---|---|---|
| `default` | Reads only | Reviewing every action yourself, sensitive work |
| `acceptEdits` | Reads, file edits, and common filesystem commands (`mkdir`, `touch`, `mv`, `cp`, etc.) | Iterating on code you're reviewing |
| `plan` | Reads, plus classifier-approved commands when auto mode is available | Exploring a codebase before changing it |
| `auto` | Everything, with background safety checks | Long tasks, reducing prompt fatigue |
| `dontAsk` | File reads inside your working directories and pre-approved tools; anything that would prompt is denied | Locked-down CI and scripts |
| `bypassPermissions` | Everything | Isolated containers and VMs only |

## 13.2 Choosing a mode

| YOU WANT TO... | START WITH | ISOLATION NEEDED |
|---|---|---|
| **Review every action yourself** | `claude --permission-mode default` | None |
| **Explore before changing anything** | `claude --permission-mode plan` | None |
| **Work hands-off with background safety checks** | `claude --permission-mode auto` | None required. A sandbox or container adds defence in depth |
| **Run in CI with an exact allowlist** | `claude -p "run the test suite" --permission-mode dontAsk --allowedTools "Bash(npm test)" "Read"` | Your CI runner |
| **Run fully unattended** | `claude -p "<prompt>" --dangerously-skip-permissions` | **Required:** a container, VM, or sandbox runtime |

Set a default for every session with `permissions.defaultMode` in `~/.claude/settings.json`, or in a project's `.claude/settings.json`. According to the docs, project settings can't make `auto` or `bypassPermissions` the starting mode, which stops a cloned repository from quietly granting itself autonomy.

> **DANGER**
>
> `bypassPermissions` means **every** action runs with no checks. Only use it inside something you're happy to throw away: a container, a VM, or a cloud sandbox with no access to your real accounts or files (Chapter 2).

## 13.3 Every settings key, A–Z

Search this table when a guide mentions a setting. Many keys only matter to administrators. Run `/config` or ask Claude (`/update-config`) to change settings safely.

| KEY | WHAT IT DOES |
|---|---|
| `advisorModel` | Pick which model answers when Claude calls the server-side advisor tool. |
| `alwaysThinkingEnabled` | Turn extended thinking off for every session by setting this to `false`. |
| `availableModels` | Restrict which models people can select for the main session, subagents, skills, and the advisor. |
| `availableModelsMatch` | Choose how `availableModels` entries match model IDs. |
| `deniedModels` | Block specific models, with or without an `availableModels` allowlist and even when that list permits them. |
| `effortLevel` | Set a default effort level for models you haven't saved a level for. |
| `enforceAvailableModels` | The `/model` picker has a **Default** option, and `default` model setting describes the model it resolves to. |
| `fallbackModel` | Name backup models for Claude Code to try, in order, when your primary model is overloaded or unavailable. |
| `fastMode` | Turn fast mode on for sessions where it's available, for interactive work like rapid iteration or live debugging where you want speed at... |
| `fastModePerSessionOptIn` | Normally, running `/fast` saves `fastMode` to a person's user settings, so fast mode is on at the start of every later session. |
| `language` | Have Claude respond in a language other than English by default. |
| `maxEffortLevel` | Cap the effort level a session can use, leaving lower levels available. |
| `model` | Set the model every new session uses, so you don't have to pick one with `/model` each time. |
| `modelOverrides` | Map Anthropic model IDs to provider-specific model IDs, such as Amazon Bedrock inference profile ARNs. |
| `modelPicker` | List the models the `/model` picker offers, in the order you write them and under labels you choose, so the picker lists the models your... |
| `modelPricing` | Report spend at the rates your organization pays instead of list price. |
| `modelSettings` | Save an effort level for each model you use. |
| `outputStyle` | Select an output style by name. |
| `promptCacheTtl` | Choose how long the prompt cache holds the main conversation. |
| `showThinkingSummaries` | See summaries of Claude's extended thinking in interactive sessions. |
| `subagentPromptCacheTtl` | Choose how long the prompt cache holds the requests Claude Code makes outside the main conversation. |
| `switchModelsOnFlag` | Choose what happens when a safety classifier flags a request: switch to the fallback model and continue, or pause so you can choose... |
| `ultracode` | Start sessions with ultracode on. |
| `allowManagedPermissionRulesOnly` | Make managed settings the only settings source of permission rules. |

| | |
|---|---|
| `autoMode` | Add your own rules to what the auto mode classifier blocks and allows. |
| `autoMode.classifyAllShell` | Send every Bash and PowerShell command through the auto mode classifier while auto mode is active. |
| `disableAutoMode` | Remove auto mode from the `Shift+Tab` cycle. |
| `permissions` | Control which tools Claude can use without asking, which ones always prompt, and which ones are blocked, and set the permission mode a… |
| `useAutoModeDuringPlan` | Choose whether Claude Code uses the auto mode classifier to review shell commands in plan mode. |
| `permissions.allow` | List the tool uses Claude Code approves without asking you. |
| `permissions.ask` | List the tool uses that prompt you for confirmation even in a permission mode that would otherwise approve them, such as `acceptEdits`… |
| `permissions.deny` | List the tool uses Claude Code blocks. |
| `permissions.additionalDirectories` | Give Claude file access to directories outside the one you started in, as additional working directories. |
| `permissions.blockReadsOutsideWorkingDirectories` | Make Claude's file tools refuse reads outside your working directories in every permission mode, including `bypassPermissions`. |
| `permissions.defaultMode` | Set the permission mode new sessions start in. |
| `permissions.disableBypassPermissionsMode` | Prevent anyone from entering `bypassPermissions` mode. |
| `skipAutoPermissionPrompt` | Skip the one-time notice describing auto mode that Claude Code shows when you first enter auto mode yourself, for example through your… |
| `skipDangerousModePermissionPrompt` | Skip the confirmation dialog Claude Code shows before a session enters `bypassPermissions` mode, whether from… |
| `sandbox` | Isolate the Bash commands Claude runs from your filesystem and network with sandboxing. |
| `sandbox.enabled` | Turn on sandboxing for Bash commands. |
| `sandbox.failIfUnavailable` | Make Claude Code exit with an error at startup when `sandbox.enabled` is `true` but the sandbox can't start, because a dependency is… |
| `sandbox.autoAllowBashIfSandboxed` | Let Claude Code run sandboxed Bash commands without a permission prompt. |
| `sandbox.excludedCommands` | Name commands that Claude Code runs outside the sandbox, such as tools that don't work under it. |
| `sandbox.allowUnsandboxedCommands` | Let Claude retry a command outside the sandbox with the `dangerouslyDisableSandbox` parameter after the sandbox blocks it. |
| `sandbox.filesystem` | Control which paths sandboxed commands can read and write. |
| `sandbox.filesystem.allowWrite` | Add paths where sandboxed commands can write, beyond the working directory, the per-user temp directory, and the directories you've… |

| | |
|---|---|
| `sandbox.filesystem.denyWrite` | Block sandboxed commands from writing to specific paths, including paths inside a directory that is otherwise writable. |
| `sandbox.filesystem.denyRead` | Block sandboxed commands from reading specific paths, such as credential files that the default read policy would otherwise expose. |
| `sandbox.filesystem.allowRead` | Re-open reading for specific paths inside a region that `denyRead` blocks, to build workspace-only read access. |
| `sandbox.filesystem.allowManagedReadPathsOnly` | Honor only the `allowRead` entries that come from managed settings, so developers can't re-open read access to paths your organization... |
| `sandbox.filesystem.disabled` | Skip filesystem isolation while keeping network isolation. |
| `sandbox.ignoreViolations` | Silence sandbox violation reports for paths you expect a command to probe and be refused, such as a tool that checks `/etc/hosts` on... |
| `sandbox.enableWeakerNestedSandbox` | Run the Linux sandbox inside an unprivileged Docker container, where bubblewrap can't mount a fresh `/proc`. |
| `sandbox.enableWeakerNetworkIsolation` | Let sandboxed commands on macOS reach the system TLS trust service, `com.apple.trustd.agent`. |
| `sandbox.allowAppleEvents` | Let sandboxed commands on macOS send Apple Events, which `open`, `osascript`, and tools that open URLs in a browser need; without it... |
| `sandbox.ripgrep` | Point the sandbox at a ripgrep binary of your own instead of the one Claude Code uses, for example when your platform needs a... |
| `sandbox.bwrapPath` | Point the sandbox at a bubblewrap binary installed outside `PATH`, such as a vendored copy on an air-gapped host. |
| `sandbox.socatPath` | Point the sandbox network proxy at a `socat` binary installed outside `PATH`. |
| `sandbox.credentials` | Declare the credential files and environment variables to protect from sandboxed commands. |
| `sandbox.credentials.files` | Protect credential files or directories from sandboxed commands. |
| `sandbox.credentials.envVars` | Protect environment variables from sandboxed commands. |
| `sandbox.credentials.allowPlaintextInject` | Allow `mask` substitution on plain HTTP requests as well as TLS-terminated HTTPS. On plain HTTP the upstream identity is unverified and... |
| `sandbox.credentials.awsPairs` | Group masked environment variables that form one AWS credential for SigV4 re-signing when your credential lives in variables with... |
| `sandbox.credentials.sigv4` | Choose what the sandbox proxy does with AWS request forms it can't re-sign: `streaming` for aws-chunked streaming uploads, `presigned`... |
| `sandbox.network` | Control which hosts, ports, and sockets sandboxed commands can reach. |
| `sandbox.network.allowUnixSockets` | List the Unix socket paths sandboxed commands can connect to on macOS. Claude Code ignores this list on Linux and WSL2, where the... |
| `sandbox.network.allowAllUnixSockets` | Let sandboxed commands connect to every Unix socket. |
| `sandbox.network.allowLocalBinding` | Let sandboxed commands on macOS listen on network ports, for example to start a dev server, and connect to any port on localhost. |
| `sandbox.network.allowMachLookup` | List additional XPC and Mach service names the macOS sandbox may look up. |

| | |
|---|---|
| `sandbox.network.allowedDomains` | Pre-allow domains for outbound traffic from sandboxed commands, so the sandbox doesn't prompt for them. |
| `sandbox.network.deniedDomains` | Block domains for outbound traffic from sandboxed commands, using the same wildcard, port, and IPv6 syntax as `allowedDomains`. |
| `sandbox.network.strictAllowlist` | Deny sandboxed commands access to hosts outside the allowlist instead of prompting for approval. |
| `sandbox.network.allowManagedDomainsOnly` | Lock the network allowlist to what managed settings define. |
| `sandbox.network.httpProxyPort` | Point the sandbox at your own HTTP proxy instead of the one Claude Code runs. |
| `sandbox.network.socksProxyPort` | Point the sandbox at your own SOCKS5 proxy instead of the one Claude Code runs. |
| `sandbox.network.tlsTerminate` | Make the sandbox proxy terminate TLS so it can read the contents of HTTPS requests. |
| `autoCompactEnabled` | Have Claude Code compact the conversation automatically when context approaches the limit. |
| `autoCompactWindow` | Set how full the context window gets before Claude Code compacts automatically. |
| `autoMemoryDirectory` | Store auto memory in a directory of your choice instead of the per-project default. |
| `autoMemoryEnabled` | Turn auto memory on or off. |
| `bashOutputMaxChars` | Set how many characters of a successful Bash or PowerShell command's output Claude receives inline. |
| `claudeMd` | Inject CLAUDE.md-style instructions as organization-managed memory without deploying a separate file. |
| `claudeMdExcludes` | Skip specific `CLAUDE.md` files when Claude Code loads memory. |
| `env` | Set environment variables for every session and for the subprocesses Claude Code starts from it. |
| `fileCheckpointingEnabled` | Have Claude Code snapshot files before each edit so `/rewind` can restore them. |
| `idleCompaction` | Idle compaction compacts a long conversation that you've stepped away from, shortly before its prompt cache expires. |
| `plansDirectory` | Choose where Claude Code stores the plan files it writes in plan mode. |
| `skillListingBudgetFraction` | Each turn, Claude sees a listing of your skills with their descriptions, and Claude Code caps that listing at a share of the context window. |
| `skillListingMaxDescChars` | Each turn, Claude sees a listing of your skills that shows each skill's `description` and `when_to_use` text. |
| `taskOutputMaxChars` | Through v2.1.276, you set this key to the number of characters of a background task's output that Claude received inline when it read... |
| `askUserQuestionTimeout` | Let an unanswered `AskUserQuestion` dialog auto-continue after a period of idle time, submitting whatever options you had already selected. |
| `autoContinueAtUsageLimit` | After a claude.ai usage limit stops your session, wait in the open session and continue the task automatically after the reset. |
| `autoScrollEnabled` | Follow new output to the bottom of the conversation in fullscreen rendering. |

| | |
|---|---|
| `axScreenReader` | Render screen-reader friendly output: flat text without decorative borders or animations. |
| `bashEditDiffEnabled` | Choose whether Claude Code records which files changed in a Git repository while a Bash command runs. |
| `companyAnnouncements` | Show your organization's announcements to users at startup. |
| `defaultShell` | Choose whether Bash or PowerShell runs the shell commands you type with the `!` prefix in the input box, the ones Claude Code runs... |
| `dialogExpiry` | Set the deadline for dialogs Claude Code forwards to a remote client, such as a Remote Control or SDK host, and for the approval dialog... |
| `editorMode` | Choose the key binding mode for the input prompt. |
| `emojiCompletionEnabled` | Show emoji suggestions when you type `:` plus a shortcode in the prompt input, and replace a completed shortcode such as `:heart:` with... |
| `fileSuggestion` | Run your own command to supply `@` file path autocomplete instead of the built-in file suggestion. |
| `footerLinksRegexes` | Render extra clickable badges in the footer below the input box when a regex matches turn output: tool results, including file contents... |
| `keybindingFlavor` | In v2.1.238 through v2.1.260, setting it to `"readline"` made `Ctrl+W` delete back to the previous whitespace instead of only the... |
| `maxProseWidth` | Cap the width of the prose in Claude's responses so lines stay readable in a wide terminal. |
| `prefersReducedMotion` | Reduce or turn off interface animations such as the spinner, shimmer, and flash effects. |
| `promptSuggestionEnabled` | Show or hide prompt suggestions, the grayed-out predictions that appear in your prompt input. |
| `respectGitignore` | Control whether the `@` file picker leaves out files that match `.gitignore` patterns. |
| `respondToBashCommands` | Choose whether Claude responds after you run a shell command with the `!` prefix in the input box. |
| `showClearContextOnPlanAccept` | When Claude finishes a plan in plan mode, it shows an approval menu. |
| `showTurnDuration` | Show or hide the turn duration message after each response, such as "Cooked for 1m 6s · done 6:05 PM". The clock after "done" shows when... |
| `spellcheck` | Underline misspelled words in the prompt input as you type, using a spell checker you install. |
| `spinnerTipsEnabled` | While Claude works, the spinner line rotates through short tips about Claude Code features, such as "Use Plan Mode to prepare for a... |
| `spinnerTipsOverride` | Add your own tips to the spinner tips that Claude Code shows while Claude works, or replace the built-in tips with yours. |
| `spinnerVerbs` | While a turn is in progress, the spinner shows a rotating verb such as "Accomplishing", "Architecting", or "Baking". Use this key to add... |
| `statusLine` | Run your own command to render a status line below the prompt with context such as the model, cost, or git branch. |
| `subagentStatusLine` | When Claude runs subagents, Claude Code lists them in a task display below the prompt, one row per subagent showing `name · description... |
| `syntaxHighlightingDisabled` | Claude Code colors code by language in the diffs, code blocks, and file previews it shows in the terminal, with its built-in... |
| `terminalProgressBarEnabled` | Some terminals can show a progress indicator on the tab or in the taskbar for the program running in them. |

| | |
|---|---|
| `terminalTitleFromRename` | Claude Code sets your terminal tab's title. |
| `theme` | Pick the color theme for the interface. |
| `timeFormat` | Choose how Claude Code writes the times it shows in the interface, such as the `done 6:05 PM` at the end of each turn duration message… |
| `timeZone` | Show the times in the interface in a time zone other than your system's. |
| `tui` | Choose the terminal UI renderer. |
| `verbose` | By default, the transcript collapses each tool call to a short summary, such as the command Claude ran and a line count of its output,… |
| `viewMode` | Set the transcript view Claude Code starts in: `"default"`, `"verbose"`, or `"focus"`. |
| `vimInsertModeRemaps` | Map two-key INSERT-mode sequences to Escape in vim editor mode. |
| `voice` | Turn on voice dictation and choose how the dictation key behaves. |
| `voiceEnabled` | Turn voice dictation on with the single Boolean form that predates the `voice` object. |
| `wheelScrollAccelerationEnabled` | Accelerate mouse-wheel scroll speed during fast scrolls in fullscreen rendering. |
| `attribution` | Customize the attribution Claude Code adds to git commits and pull requests. |
| `includeCoAuthoredBy` | Use `attribution` instead, which replaces this key and lets you change or hide the commit trailer, the pull request text, and the… |
| `includeGitInstructions` | Claude Code gives Claude two git-related pieces of context: its built-in instructions for how to write commits and pull requests, in the… |
| `prUrlTemplate` | Point the PR links Claude Code renders, in the footer badge and in tool-result summaries, at an internal code-review tool instead of… |
| `attribution.commit` | Set the attribution text Claude Code adds to git commits, including any trailers. |
| `attribution.pr` | Set the attribution text Claude Code adds to pull request descriptions. |
| `attribution.sessionUrl` | Choose whether Claude Code appends the claude.ai session link when it commits or opens a pull request from a cloud or Remote Control… |
| `allowedHttpHookUrls` | Limit which URLs HTTP hooks can target. |
| `allowManagedHooksOnly` | Restrict hook execution to hooks your organization deploys. |
| `disableAllHooks` | Turn off hooks, any custom status line, and any custom file suggestion command. |
| `disableWorkflows` | Turn off dynamic workflows and the bundled workflow commands for everyone your settings reach, such as an organization through managed… |
| `enableWorkflows` | Turn dynamic workflows on or off for yourself when your plan's default isn't what you want. |
| `hooks` | Run your own commands, prompts, agents, HTTP requests, or MCP tools as hooks at points in Claude Code's lifecycle, such as before a tool… |
| `httpHookAllowedEnvVars` | An HTTP hook can put the value of an environment variable into a request header, for example an `Authorization: Bearer $HOOK_TOKEN`… |
| `workflowKeywordTriggerEnabled` | Choose whether typing the keyword `ultracode` in a prompt triggers a dynamic workflow. |
| `workflowSizeGuideline` | Set the agent count Claude aims for in the dynamic workflows it writes. |
| `disableBundledSkills` | Turn off the skills and workflows included with Claude Code. |
| `disableSkillShellExecution` | Turn off inline shell execution for ``!`...` `` and ``` ```! ``` blocks in skills and custom commands from user, project, plugin, or… |

| | |
|---|---|
| `skillOverrides` | Hide or collapse a skill without editing its `SKILL.md`. |
| `syncClaudeAiSkills` | Turn off the download of the skills enabled for your claude.ai account. |
| `syncClaudeAiPlugins` | Turn off the download of the plugins enabled for your claude.ai account. |
| `allowedChannelPlugins` | Choose which channel plugins can push messages into sessions in your organization. |
| `blockedMarketplaces` | Block plugin marketplace sources for your organization. |
| `channelsEnabled` | Allow channels for your organization. |
| `disableCommandPluginSources` | Block the `command` plugin source, which installs a plugin by running a marketplace-declared command on the user's machine. |
| `pluginSuggestionMarketplaces` | Name the marketplaces whose plugins can appear as contextual install suggestions, in spinner tips and pinned at the top of the `/plugin` ... |
| `pluginTrustMessage` | Add your organization's own text to the plugin trust warning Claude Code shows before installation, for example to confirm that plugins... |
| `strictKnownMarketplaces` | Restrict which plugin marketplace sources people in your organization can add and install plugins from. |
| `strictPluginOnlyCustomization` | Block skills, agents, hooks, and MCP servers from user and project sources, so they can come only from plugins or managed settings. |
| `strictPluginOnlyCustomization.skills` | Lock the `skills` surface. |
| `strictPluginOnlyCustomization.agents` | Lock the `agents` surface. |
| `strictPluginOnlyCustomization.hooks` | Lock the `hooks` surface. |
| `strictPluginOnlyCustomization.mcp` | Lock the `mcp` surface. |
| `enabledPlugins` | Turn individual plugins on or off, keyed by `plugin-name@marketplace-name`. |
| `extraKnownMarketplaces` | Register additional plugin marketplaces by name, so that people who open the repository, or everyone your managed settings reach, get... |
| `pluginConfigs` | Store the non-sensitive answers you give a plugin's `userConfig` configuration dialog, keyed by plugin ID. Claude Code writes this key... |
| `prependPlugins` | List the managed plugins whose mods run before every mod a user installs, in the listed order. |
| `appendPlugins` | List the managed plugins whose mods run after every mod a user installs, in the listed order. |
| `allowAllClaudeAiMcps` | Load the claude.ai connectors Claude Code fetches itself alongside a deployed `managed-mcp.json`. |
| `allowClaudeInChromeWithManagedMcp` | Let the built-in Claude in Chrome server run alongside a deployed `managed-mcp.json`. |
| `allowedMcpServers` | Allowlist the MCP servers people can add. |
| `allowManagedMcpServersOnly` | Make the managed allowlist the only one that applies. |
| `deniedMcpServers` | Block specific MCP servers. |
| `disableClaudeAiConnectors` | Turn off the claude.ai MCP connectors Claude Code fetches itself, so it neither fetches nor connects them. |
| `disabledMcpjsonServers` | Reject specific servers defined in a project's `.mcp.json` file so Claude Code never connects them or asks you to approve them. |

| | |
|---|---|
| `enableAllProjectMcpServers` | Approve every MCP server defined in project `.mcp.json` files without a prompt. |
| `enabledMcpjsonServers` | Approve specific servers defined in project `.mcp.json` files so Claude Code connects them without asking. |
| `managedMcpServers` | Provide remote MCP servers to every user from managed settings. |
| `agent` | Run the main thread as a named subagent, so Claude Code applies that subagent's system prompt, tool restrictions, and model to your session. |
| `crossSessionInbound` | Choose what this session does with messages arriving from your other Claude Code sessions. |
| `disableAgentView` | Turn off background agents and agent view: `claude agents`, `--bg`, `/background`, and the on-demand supervisor. |
| `isolatePeerMachines` | Require your explicit approval before Claude's `SendMessage` reaches one of your sessions beyond this machine; see Require approval for... |
| `processWrapper` | On macOS and Linux, place a corporate launcher command in front of the background processes Claude Code starts. |
| `teammateMode` | Choose where Claude Code shows agent team teammates: inside your main terminal pane, or in split panes when your terminal supports them. |
| `worktree` | Configure how Claude Code creates and manages git worktrees for `--worktree`, the `EnterWorktree` tool, and isolated subagents and... |
| `worktree.baseRef` | Choose which ref new worktrees branch from. `"fresh"` branches from `origin/` for a clean tree matching the remote; `"head"` branches... |
| `worktree.symlinkDirectories` | Symlink directories from the main repository into each worktree so you don't duplicate large directories on disk. |
| `worktree.sparsePaths` | Check out only the listed directories in each worktree through git sparse-checkout. |
| `worktree.bgIsolation` | Choose how background sessions isolate their file edits. |
| `worktree.location` | Choose the folder on a remote machine where Desktop SSH sessions create their worktrees, instead of `/.claude/worktrees/`. |
| `agentPushNotifEnabled` | Allow Claude to send a push notification to your phone when it decides one is worth sending, for example when a long task finishes. |
| `awaySummaryEnabled` | Show a one-line session recap when you return to the terminal after a few minutes away. |
| `disableArtifact` | Use `enableArtifact` instead to turn off the Artifact tool, which publishes session output as a private web page on claude.ai. |
| `disableDeepLinkRegistration` | Stop Claude Code from registering the `claude-cli://` protocol handler with the operating system, which it otherwise does after you send... |
| `disableDesktopLocalSessions` | Turn off Code sessions that run on the device in the desktop app, for deployments where developers should work on remote machines over... |
| `disableRemoteControl` | Turn off Remote Control: Claude Code then refuses `claude remote-control`, the `--remote-control` flag, auto-start, and the in-session... |
| `enableArtifact` | Turn off the Artifact tool, which publishes session output as a private web page on claude.ai. |
| `inputNeededNotifEnabled` | Get a push notification on your phone when a permission prompt or question is waiting for your input. |
| `preferredNotifChannel` | Choose how Claude Code notifies you when a task completes or a permission prompt is waiting. |
| `remote.defaultEnvironmentId` | Pick the default cloud environment for cloud sessions you create from the CLI, such as with `claude --cloud`. |

| | |
|---|---|
| `remoteControlAtStartup` | Connect Remote Control automatically when each interactive session starts, instead of waiting for `/remote-control`. |
| `sshConfigs` | Add SSH connections to the Desktop environment dropdown. |
| `sshHostAllowlist` | Limit the hosts a Desktop SSH session can connect to. |
| `allowedProviders` | List the services a machine may reach Claude through, such as the Anthropic API, Amazon Bedrock, or an LLM gateway. |
| `apiKeyHelper` | Run your own command to produce the credential Claude Code sends with model requests. |
| `awsAuthRefresh` | Run your own command, such as `aws sso login`, to refresh the credentials in your `.aws` directory when the ones Claude Code has for… |
| `awsCredentialExport` | Run your own command that prints AWS credentials as JSON, so Claude Code can call Amazon Bedrock with credentials that don’t live in… |
| `forceLoginMethod` | Restrict which kind of account people can log in with. |
| `forceLoginGatewayUrl` | Set the gateway URL the `/login` Cloud gateway screen connects to, so people reach your cloud gateway without typing its address. |
| `forceLoginOrgUUID` | From a managed source, require claude.ai account logins to belong to one Anthropic organization, given as a single UUID, or to any of… |
| `gatewayInternalNetworks` | Declare the public IPv4 blocks that your organization numbers its internal network from, so `/login` accepts a cloud gateway there. |
| `gcpAuthRefresh` | Run your own command to refresh Google Cloud Application Default Credentials when Claude Code finds they’ve expired or can’t be loaded,… |
| `otelHeadersHelper` | Run your own command to generate the headers Claude Code sends with OpenTelemetry exports, for backends whose tokens rotate. |
| `autoUpdatesChannel` | Choose which release channel background auto-updates and `claude update` follow. |
| `minimumVersion` | Keep background auto-updates and `claude update` from installing any version below this one, so moving to the `"stable"` channel doesn’t… |
| `requiredMaximumVersion` | Set the newest Claude Code version your organization allows to start. |
| `requiredMinimumVersion` | Set the oldest Claude Code version your organization allows to start. |
| `browserExternalPageTools` | Stop Claude from using its tools to read or act on external pages in the desktop app’s Browser pane. |
| `disableBrowserExternalNavigation` | Turn off external browsing in the desktop app’s Browser pane for people and Claude alike. |
| `disableMobileSimulatorTools` | Block Claude’s tools for the desktop app’s iOS Simulator pane. |
| `cleanupPeriodDays` | Set how many days Claude Code keeps session transcripts and other application data before deleting them. |
| `desktopSessionCleanupPeriodDays` | Set an age limit in days for the transcripts of sessions you started or most recently continued in Claude Desktop or Cowork. |
| `feedbackDrafts` | Control Claude-drafted feedback: whether Claude can queue feedback drafts for you to review, and whether Claude Code shows a card when… |
| `feedbackSurveyRate` | Set the probability that the session quality survey appears when a session is eligible for it. |
| `skipWebFetchPreflight` | Skip the WebFetch domain safety check, which sends each requested hostname to `api.anthropic.com` before fetching. |

| | |
|---|---|
| `disableSideloadFlags` | Reject the `--plugin-dir`, `--plugin-url`, `--agents`, and `--mcp-config` CLI flags at startup, which users could otherwise pass to... |
| `forceRemoteSettingsRefresh` | Block CLI startup until Claude Code has freshly fetched server-managed settings. |
| `managedSourcesBehavior` | Choose whether Claude Code applies only the highest-priority managed source your organization delivers, or combines every admin source... |
| `parentSettingsBehavior` | Choose whether Claude Code applies managed settings supplied by an embedding host process, such as the Agent SDK or an IDE extension,... |
| `policyHelper` | Run an executable you deploy that computes managed settings at startup, so you can derive policy from device posture, identity, or a... |
| `policyHelper.path` | Name the helper executable Claude Code runs. |
| `policyHelper.timeoutMs` | Set how long Claude Code waits for the helper before treating the run as failed. |
| `policyHelper.refreshIntervalMs` | Have Claude Code re-run the helper in the background on an interval so policy changes reach a running session. |
| `wslInheritsWindowsSettings` | Have Claude Code on WSL read managed settings from the Windows policy chain, with HKLM and the Windows managed settings file taking... |
| `autoConnectIde` | Connect to a running IDE automatically when you start Claude Code from an external terminal. |
| `autoInstallIdeExtension` | Install the Claude Code IDE extension automatically when you run Claude Code from a VS Code terminal. |
| `claudeInChromeDefaultEnabled` | Start every interactive CLI session with Chrome integration on, without passing `--chrome` each time. |
| `copyFullResponse` | Make `/copy` copy the full response every time, without showing the picker. |
| `copyOnSelect` | Copy text to your clipboard automatically when you finish selecting it with the mouse in fullscreen rendering or agent view. |
| `defaultToAgentsView` | Open agent view instead of a new conversation when you run `claude` with no arguments. |
| `diffTool` | Choose where Claude Code shows the diff of an `Edit` or `Write` change it proposes when a VS Code or JetBrains IDE is connected:... |
| `externalEditorContext` | When you press `Ctrl+G`, Claude Code opens the prompt you're typing in your external editor. |
| `leftArrowOpensAgents` | Press `←` on an empty prompt to background the session and open agent view. |
| `permissionExplainerEnabled` | Through v2.1.256, you could press `Ctrl+E` on a Bash or PowerShell permission prompt to see a model-generated explanation of the... |
| `prStatusFooterEnabled` | Show a badge in the prompt footer for the current branch's open pull request or merge request, with a colored underline that shows its... |
| `teammateDefaultModel` | Through v2.1.233, you set this key to the model for agent team teammates your prompt didn't name a model for: an alias such as... |

CHAPTER 14

# Autonomy Recipes

Copy-ready setups for the most common hands-off scenarios. Each recipe lists the risk level and the guardrails you must keep.

| RECIPE | SETUP | RISK | MUST-KEEP GUARDRAILS |
|---|---|---|---|
| **Overnight refactor in a container** | Dev container or VM, `--dangerously-skip-permissions`, a git branch | Medium | No production credentials in the box. Review the diff in the morning |
| **Daily docs or report job** | `claude -p` from cron, `dontAsk`, an allowlist of exact commands | Low | Read-only tokens. Output to a file, not straight to publishing |
| **PR auto-fix bot** | CI runner, allowlist of test and lint commands, a scoped GitHub token | Medium | Branch protection on `main`. A human merges |
| **n8n AI agent with tools** | Agent node with minimal tools, approval step for sending or deleting | Medium | Error workflow, rate limits, logging |
| **Notion assistant** | Integration shared with one teamspace only | Low | No delete permission. Weekly audit |
| **Browser agent** | Separate browser profile, no saved passwords | High | Watch it live. Never let it handle payments unattended |
| **Trading bot** | Paper account only (Vol. 10) | High | Kill switch, loss limits, separate keys |

## 14.1 Golden rules of autonomy

- [ ] Autonomy goes in a box: a container, VM, or separate account
- [ ] Credentials are scoped to the task and set to expire
- [ ] Anything that can’t be undone gets a human approval step
- [ ] Everything is logged, and logs are reviewed
- [ ] There’s a tested kill switch for every agent
- [ ] Start with a short leash and lengthen it only after a clean track record

CHAPTER 15

# Incident Response: When an Agent Goes Wrong

Sooner or later an agent will do something you didn't expect. A calm, practised routine turns a disaster into an inconvenience.

## STOP, CONTAIN, RECOVER

1. **Stop it.**
   Press `Esc` or `Ctrl` + `C`, stop the container or VM, deactivate the n8n workflow, or use `/stop` for a background session.

2. **Cut its access.**
   Revoke or rotate any tokens it held: GitHub, cloud, Notion, MCP connectors, API keys.

3. **Assess.**
   Check `git status` and `git log`, cloud consoles, sent mail, and billing pages. What changed?

4. **Restore.**
   `git revert` or restore from a branch or backup. Use `/rewind` for local file edits. Restore databases from snapshots.

5. **Notify.**
   If other people's data or money was affected, tell them, and the relevant service.

6. **Learn.**
   Write down what permission made it possible, then tighten that rule or add a hook or deny rule.

**Emergency card:** Esc / Ctrl+C → kill container → revoke tokens → check git, billing, and sent mail → revert → tighten rules.

# Your Autonomy Setup Checklist

- [ ] Agents run in a dev container, VM, cloud box, or separate user account
- [ ] Separate GitHub account or app for agents; fine-grained token with full repo permissions
- [ ] Hard spend limits set on every AI and cloud account
- [ ] Everything in Git; `main` protected by status checks; auto-merge allowed
- [ ] `~/.claude/settings.json`: `defaultMode` set to `auto` or `bypassPermissions`, plus `skipDangerousModePermissionPrompt`
- [ ] Claude Desktop: **Allow bypass permissions mode** on; Cowork: **Automatically approve**
- [ ] Every connector tool set to **Always allow**
- [ ] Cursor: **Run Everything** (or Auto-review with allow instructions)
- [ ] VS Code: `chat.tools.global.autoApprove` on, or Autopilot
- [ ] Codex `config.toml`: `approval_policy = "never"`, `sandbox_mode = "danger-full-access"`; Gemini alias with `--yolo`
- [ ] n8n: API key, instance-level MCP on, workflows enabled for MCP; `NODES_EXCLUDE="[]"` if self-hosted
- [ ] Notion: MCP connected; API connection with all capabilities and top-level pages shared
- [ ] Unattended runs have `--max-turns` and `--max-budget-usd` limits

# Glossary

| | |
|---|---|
| **Allowlist** | A list of pre-approved actions an agent may take without asking. |
| **Auto mode** | Claude Code's permission mode in which a classifier reviews actions instead of a person. |
| **Bypass permissions** | A mode that skips approval prompts entirely (`--dangerously-skip-permissions` in Claude Code). |
| **Classifier** | A second AI model that judges whether an agent's action is safe and matches the request. |
| **Containment** | Limiting what an agent can reach (a container, separate accounts, budgets) instead of limiting what it may do. |
| **Critical path** | A folder, such as the root, your home folder, or the working directory, that Claude Code protects from deletion in every mode. |
| **Dev container** | A Docker container configured as a development environment, used to isolate agents. |
| **Fine-grained token** | A GitHub access token limited to chosen repositories and permissions. |
| **Headless / print mode** | Running an agent non-interactively from a script, such as `claude -p`. |
| **Prompt injection** | Instructions hidden in content an agent reads, meant to hijack its behavior. |
| **Ruleset** | GitHub settings that protect branches, with an optional bypass list. |
| **Sandbox** | An isolated environment limiting the files and network a command can reach. |
| **Sudo mode** | GitHub's re-authentication step for sensitive actions in the web interface. |
| **YOLO mode** | Informal name for fully automatic approval in tools like Gemini CLI, Codex, and Copilot CLI. |